

Striver SDE Sheet - Linked List and Arrays

Quick DSA Revision Notes

Revision format: Approach -> core idea -> time and space complexity. Focus on the optimal pattern before interviews.

1. Rotate a Linked List Right by K

Approach	Core idea	Time	Space
Brute force	Move the last node to the front, one rotation at a time.	$O(N \cdot K)$	$O(1)$
Optimal	Find length N, set $k \% = N$, connect tail to head to form a circle, move to the new tail at $N - k - 1$, set newHead = newTail.next and break the circle.	$O(N)$	$O(1)$

Revision key: Key: $k \% = N$ avoids unnecessary full rotations.

2. Clone Linked List with Random Pointer

Approach	Core idea	Time	Space
Hash map	Create original-to-copy mapping, then assign next and random pointers in a second pass.	$O(N)$	$O(N)$
Optimal - Interweaving	Insert each copy after its original, set copy.random = original.random.next, then separate the two lists.	$O(N)$	$O(1)$ auxiliary

Revision key: Three phases: insert copies, assign random links, separate lists.

3. Three Sum

Approach	Core idea	Time	Space
Brute force	Check every triplet and use a set to remove duplicates.	$O(N^3)$	Output/set space
Better - Hashing	Fix i and j, then search for $-(a[i] + a[j])$ using a hash set.	$O(N^2)$	$O(N)$
Optimal - Sort + two pointers	Sort, fix i, then use left and right pointers. Move based on the sum and skip duplicates after finding a valid triplet.	$O(N^2)$	$O(1)$ excluding output

Revision key: Key: Skip duplicate i, left and right values.

4. Trapping Rainwater

Approach	Core idea	Time	Space
Brute force	For every index, search the maximum height on its left and right.	$O(N^2)$	$O(1)$
Better	Precompute prefixMax and suffixMax arrays. Water = $\min(\text{leftMax}, \text{rightMax}) - \text{height}[i]$.	$O(N)$	$O(N)$
Optimal - Two pointers	Maintain left, right, leftMax and rightMax. Process the side with the smaller boundary because that side determines the water level.	$O(N)$	$O(1)$

Revision key: Formula: $\text{water}[i] = \min(\text{maxLeft}, \text{maxRight}) - \text{height}[i]$.

5. Remove Duplicates from Sorted Array

Approach	Core idea	Time	Space
Brute force	Insert values into a set, then copy them back.	$O(N \log N)$	$O(N)$
Optimal - Two pointers	Keep i at the last unique value. For j from 1 onward, when $\text{arr}[j]$ differs from $\text{arr}[i]$, increment i and copy $\text{arr}[j]$. Return i+1.	$O(N)$	$O(1)$

Revision key: Reason: In a sorted array, equal values are adjacent.

6. Maximum Consecutive Ones

Approach	Core idea	Time	Space
Optimal	Increase current count for every 1 and update the maximum. Reset current count to zero whenever a 0 appears.	$O(N)$	$O(1)$

Revision key: Pattern: Running count with reset.